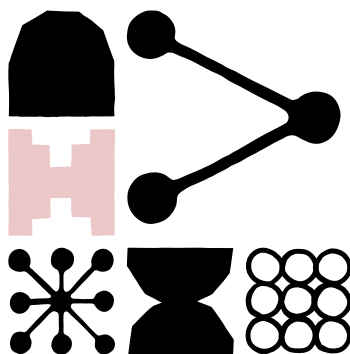


借助 Agent Skills，让 Agent 胜任现实世界的任务

Equipping agents for the real world with Agent Skills

Anthropic · 2025-10-16 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



Claude 很强大，但真实工作需要操作性知识和组织上下文。我们推出 Agent Skills，让你通过文件和文件夹，以一种新方式构建专门的 Agent。

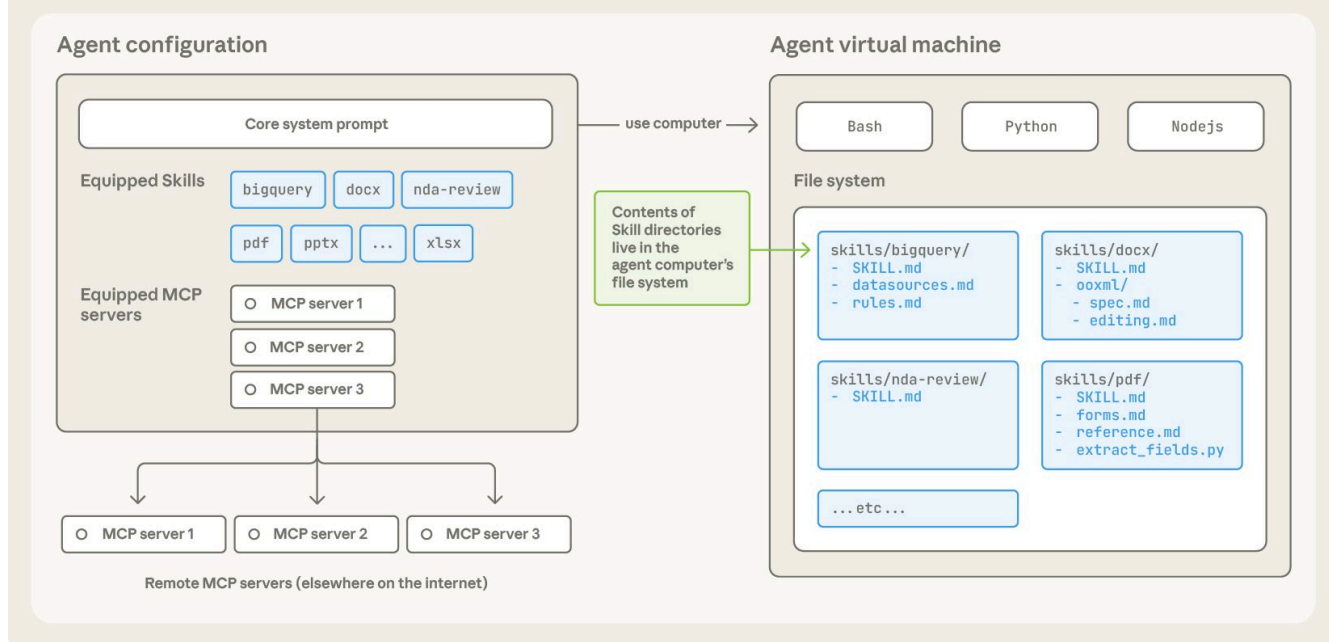
更新：我们已将 Agent Skills 发布为开放标准，以支持跨平台移植。(2025 年 12 月 18 日)

随着模型能力提升，我们现在可以构建能够与完整计算环境交互的通用 Agent。例如，Claude Code 可以利用本地代码执行和文件系统，完成跨领域的复杂任务。但随着这些 Agent 变得更强大，我们也需要组合性、可扩展性和可移植性更好的方式，为它们提供领域专长。

因此，我们创建了 **Agent Skills**：包含指令、脚本和资源的有组织的文件夹，Agent 可以动态发现并加载它们，以便在特定任务上表现得更好。Skills 将你的专业知识打包成 Claude 可组合使用的资源，以此扩展 Claude 的能力，把通用 Agent 变成符合你需求的专门 Agent。

为 Agent 构建技能，就像为新员工编写入职指南。现在，任何人都可以通过记录并分享自己的操作性知识，用可组合的能力使 Agent 具备专长，而无需为每一种用例单独构建零散、定制的 Agent。本文将解释 Skills 是什么，展示其工作方式，并分享构建技能的最佳实践。

Agent + Skills + Virtual Machine



A skill is a directory containing a SKILL.md file that contains organized folders of instructions, scripts, and resources that give agents additional capabilities.

技能的构成

为了了解 Skills 的实际工作方式，我们来看一个真实例子：为 Claude 最近推出的文档编辑能力提供支持的其中一项技能。Claude 已经掌握了大量理解 PDF 的知识，但直接操作 PDF 的能力仍然有限，例如填写表单。这个 PDF 技能让我们能够赋予 Claude 这些新能力。

最简单的技能，就是一个包含 `SKILL.md file` 的目录。这个文件必须以 YAML 前置元数据开头，其中包含必填的 `name` 和 `description`。启动时，Agent 会将所有已安装技能的 `name` 和 `description` 预加载进系统提示词。

这些元数据是**渐进式披露的第一层**：它只提供足以让 Claude 判断何时使用各项技能的信息，而不将完整技能加载到上下文中。文件的正文构成**第二层**详细信息。如果 Claude 认为某项技能与当前任务相关，就会读取其完整的 `SKILL.md`，将技能加载到上下文中。

A simple SKILL.md file

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and tables,
merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
... python
from pypdf import PdfReader, PdfWriter

# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")

...
```

A SKILL.md file must begin with YAML Frontmatter that contains a file name and description, which is loaded into its system prompt at startup.

随着技能变得复杂，其上下文可能多到无法容纳在单个 `SKILL.md` 中，也可能有些上下文仅适用于特定场景。这时，可以把额外文件放入技能目录，并在 `SKILL.md` 中按名称引用它们。这些链接到的附加文件构成详细信息的第三层及更深层，Claude 可以只在需要时才去浏览和发现它们。

在下方的 PDF 技能中，`SKILL.md` 引用了两个附加文件：`reference.md` 和 `forms.md`。技能作者选择将它们与核心的 `SKILL.md` 一同打包。将表单填写说明移至单独的文件 `forms.md` 后，技能作者就能让核心内容保持精简，并相信 Claude 只会在填写表单时读取 `forms.md`。

Bundling additional content

pdf/SKILL.md

YAML Frontmatter

```
---
name: pdf
description: Comprehensive PDF toolkit for extracting text and
tables, merging/splitting documents, and filling-out forms.
---
```

Markdown

Overview

This guide covers essential PDF processing operations using Python libraries and command-line tools. For advanced features, JavaScript libraries, and detailed examples, see `./reference.md`. If you need to fill out a PDF form, read `./forms.md` and follow its instructions.

Quick Start

```
... python
from pypdf import PdfReader, PdfWriter
```

Read a PDF

```
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")
```

Extract text

```
text = ""
for page in reader.pages:
    text += page.extract_text()
```

pdf/reference.md

PDF Processing Advanced Reference

This document contains advanced PDF processing features, detailed examples, and additional libraries not covered in the main skill instructions.

pypdfium2 Library (Apache/BSD License)

Overview

pypdfium2 is a Python binding for PDFium (Chromium's PDF library). It's excellent for fast PDF rendering, image generation, and serves as ...

pdf/forms.md

If you need to fill out a PDF form, first check to see if the PDF has fillable form fields. Run this script from this file's directory:

```
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the "Fillable
fields" or "Non-fillable fields" and follow those
instructions.
```

Fillable fields

If the PDF has fillable form fields:

```
- Run this script from this file's directory:
`python scripts/extract_form_field_info.py <input.pdf>
<fields.json>`.
...
```

You can incorporate more context (via additional files) into your skill that can then be triggered by Claude based on the system prompt.

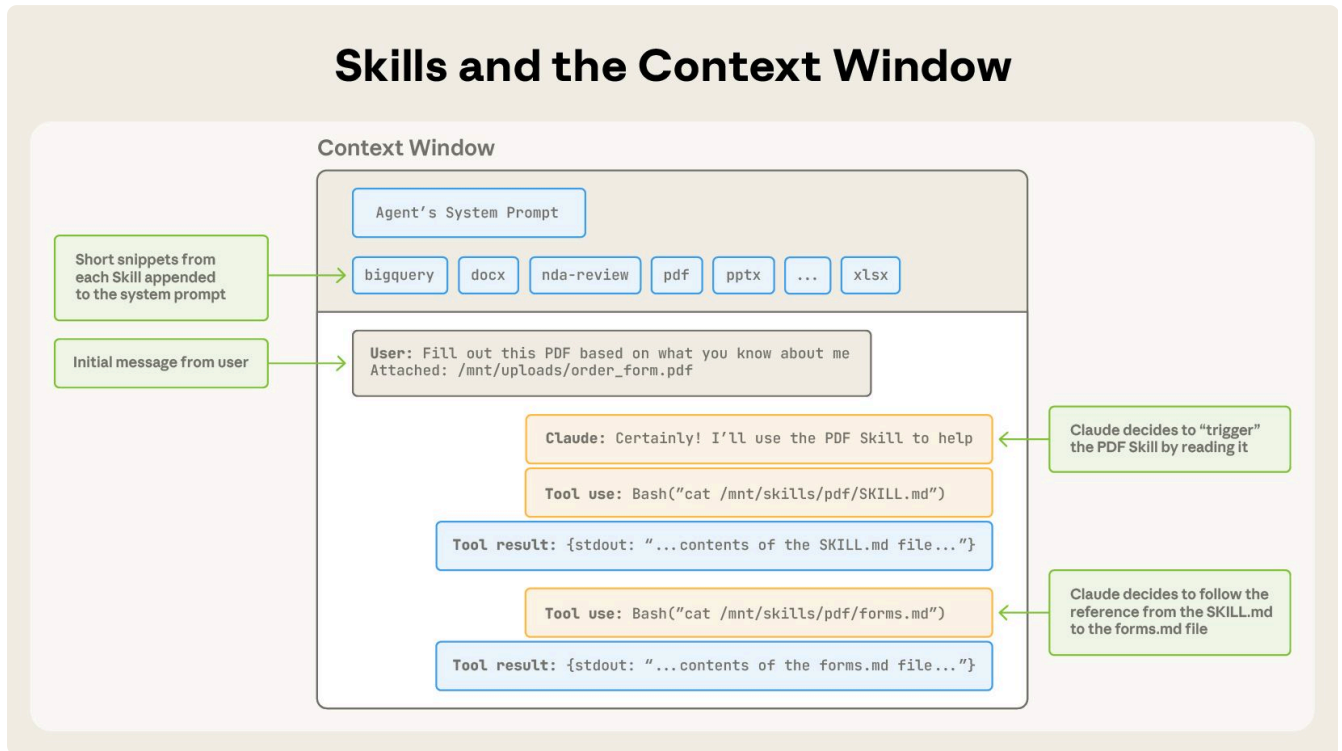
渐进式披露是使 Agent Skills 灵活且可扩展的核心设计原则。就像一本组织良好的手册，先有目录，再有具体章节，最后附上详细附录，技能让 Claude 仅在需要时加载信息：

Level	File	Context Window	# Tokens
1	SKILL.md Metadata (YAML)	Always loaded	~100
2	SKILL.md Body (Markdown)	Loaded when Skill triggers	<5k
3+	Bundled files (text files, scripts, data)	Loaded as-needed by Claude	unlimited*

拥有文件系统和代码执行工具的 Agent，在处理某个具体任务时，不必将整个技能都读入上下文窗口。这意味着，技能中能够打包的上下文量实际上没有上限。

Skills 与上下文窗口

下图展示了用户消息触发某个技能时，上下文窗口如何变化。



Skills are triggered in the context window via your system prompt.

图中的操作顺序如下：

1. 起初，上下文窗口中包含核心系统提示词、每个已安装技能的元数据，以及用户的初始消息；
2. Claude 调用 Bash 工具读取 `pdf/SKILL.md` 的内容，从而触发 PDF 技能；
3. Claude 选择读取该技能附带的 `forms.md` 文件；
4. 最后，在从 PDF 技能中加载相关指令后，Claude 开始推进用户任务。

Skills 与代码执行

技能还可以包含代码，让 Claude 自行决定何时将其作为工具执行。

大语言模型擅长许多任务，但某些操作更适合通过传统代码执行来完成。例如，通过生成 token 来对列表排序，远比直接运行排序算法昂贵。除了效率，许多应用还需要只有代码才能提供的确定性与可靠性。

在我们的例子中，PDF 技能包含一个预先编写好的 Python 脚本，用于读取 PDF 并提取所有表单字段。Claude 可以运行这个脚本，而无需将脚本或 PDF 加载到上下文中。由于代码具有确定性，这套工作流也具有一致性和可重复性。

Bundling executable scripts

pdf/forms.md

```
If you need to fill out a PDF form, first check
to see if the PDF has fillable form fields.
Run this script from this file's directory:
`python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the
"Fillable fields"
or "Non-fillable fields" and follow those
instructions.

# Fillable fields If the PDF has fillable form
fields:
- Run this script from this file's directory:
`python ./extract_fields.py
<input.pdf> <fields.json>`.
...
```

pdf/extract_fields.py

```
from pypdf import PdfReader

def write_field_info(pdf_path: str, output_path: str):
    """Extract form fields from PDF and store as JSON."""
    reader = PdfReader(pdf_path)
    fields = get_fields(reader)
    with open(output_path, "w") as f:
        json.dump(fields, f)

# ... omitted ...

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print(f"Usage: python {sys.argv[0]} <pdf_path> <output_json_path>")
        sys.exit(1)
    write_field_info(sys.argv[1], sys.argv[2])
```

Skills can also include code for Claude to execute as tools at its discretion based on the nature of the task.

开发与评测技能

以下建议有助于你开始编写和测试技能：

- **从评测开始：**让 Agent 执行具有代表性的任务，观察它们在哪些地方遇到困难或需要更多上下文，从而找出具体的能力缺口。然后逐步构建技能，弥补这些不足。
- **为扩展规模设计结构：**当 `SKILL.md` 文件变得臃肿难用时，将内容拆分到不同文件中，再通过引用关联起来。如果某些上下文适用于互斥场景，或很少一起使用，将读取路径分开可以减少 token 用量。最后，代码既可以作为可执行工具，也可以作为文档。应明确说明 Claude 是应当直接运行脚本，还是将其读入上下文作为参考。
- **从 Claude 的视角思考：**监测 Claude 在真实场景中如何使用你的技能，并根据观察迭代：留意意外的执行轨迹，以及对某些上下文的过度依赖。尤其要重视技能的 `name` 和 `description`。Claude 会据此决定是否当前任务触发该技能。
- **与 Claude 一起迭代：**当你与 Claude 共同处理任务时，请它将成功的方法和常见错误记录为技能中可复用的上下文与代码。如果它在使用技能完成任务时偏离了方向，就请它反思哪里出了问题。这个过程会帮助你发现 Claude 实际需要什么上下文，而不必一开始就试图预判全部需求。

使用 Skills 时的安全考量

Skills 通过指令和代码为 Claude 提供新能力。这使它们十分强大，但也意味着，恶意技能可能给使用环境引入漏洞，或指使 Claude 窃取并外传数据、执行非预期操作。

我们建议只从可信来源安装技能。如果要安装来自可信度较低来源的技能，应在使用前进行全面审查。首先阅读技能所附文件的内容，了解它具体做什么，尤其关注代码依赖，以及图片、脚本等附带资源。同样，也要留意技能中要求 Claude 连接可能不可信的外部网络来源的指令或代码。

Skills 的未来

目前，Claude.ai、Claude Code、Claude Agent SDK 和 Claude Developer Platform 均已支持 Agent Skills。

未来几周，我们将继续添加功能，支持 Skills 从创建、编辑、发现、分享，到使用的完整生命周期。我们尤其期待 Skills 帮助组织和个人与 Claude 分享上下文及工作流的机会。我们还将探索 Skills 如何通过教会 Agent 涉及外部工具和软件的更复杂工作流，与模型上下文协议（MCP）服务器相互补充。

更长远地看，我们希望让 Agent 能够自行创建、编辑和评测 Skills，将自身的行为模式固化为可复用能力。

Skills 是一个简单的概念，采用的格式也同样简单。这种简洁性让组织、开发者和最终用户更容易构建定制 Agent，并赋予它们新能力。

我们期待看到大家用 Skills 构建出什么。阅读 Skills 文档与实用示例集，即刻开始。

致谢

本文由 Barry Zhang、Keith Lazuka 和 Mahesh Murag 撰写，他们都非常喜欢文件夹。特别感谢 Anthropic 内部众多推动、支持并构建 Skills 的同事。